

1. product strategy case (e.g. coffee shop loyalty program case), 2. the ETF/finance reading materials, 3. Math calculations - e.g. Trade settlement risk

For super day afternoon (if you pass morning): more system design examples: 1. Design a MACS system 2. Design an alternative for processing automated scripts from manual traders

Overview

Coffee Shop loyalty Casing (product)

- New york
- Stagnant sales
- Implement a loyalty program in order to grow sales
- Clarifying questions
- Two locations in brooklyn
- Two locations in manhattan
- Coffee sells for around \$5
- Coffee margin around \$3
- Baked goods sell for around \$3
- Baked goods margin around \$2

To prep:

- Clarifying questions
 - What is the business?
 - Is it a chain or a single store? What is the positioning of the store?
 - What is being sold?
 - Do they have any data around the drop in growth?
 - Ex. Timeframe
 - Is there a specific store?
 - What type of customers is typical at the store?
 - Is sales the only goal?
 - Is there any constraints to what we can do? (technical)
 - Ex. is there an existing POS system?
 - Is there any marketing budget?
- Define the goal
 - Goal: Revenue = # of customers x frequency of visit x average order/customer
 - Where is the cause of the stagnant sales?
 - If the problem is **traffic (not enough new customers)**: competition, location/not enough foot traffic, seasonality, not marketed enough
 - If the problem is **frequency (not enough repeat customers)**: are regulars coming less often? Retention issues?

- If the problem is **average order size**: what do customers buy, what are our pricing/bundling
 - Do a margin calculation from the data given
 - Coffee sells for around \$5
 - Coffee margin around \$3
 - Baked goods sell for around \$3
 - Baked goods margin around \$2
- Segment customers: regulars, occasional, tourists, commuters
- Loyalty program options:
 - stamp card
 - points
 - app
 - phone-number system
 - subscription
 - tiers
- Expected value of program
- Cost of rewards
- Fraud / abuse risk
- Operational friction
- Go-to-market plan
- Success metrics
-

ETF finance reading materials, Aug 2014

- What happened on Aug 2014, explain why price deanchored from the snp 500
 - Understanding what are ETFs
 - Types of pricing
 - Role of the Market maker
- Whats a flash crash in stock market
- LULP questions
- Strong understanding of material

Math Calc, trade settlement risk

- Related to ETF reading material
 - Such as calculating trade settlement risk, related to the financial terms
 - Where one party fails their obligation, but the other party doesn't
 - Calculate the associated cost with that, such as asset cost, delays and timing cost
- Reasoning question
-

MACS system + Trade settlement risk system

MACS

- Market Access Control System

-

Design a system to identify abnormal data and prevent risky behavior in the communication system between market maker and central exchange.

-

-

Automated script (design an alternative for processing manual scripts for traders)

- Manual traders have written macro to automate the manual part of the system
- Design something that's safe and governed
- Ask clarifying question
- Define goal: speed vs consistency
- Input outputs, datafields, for scripts, and the market
- Users, guardrails
- Metrics, what determines success
 - What are different types of risk, and how to identify and mitigate

Coffee Shop Loyalty Program

Coffee Shop loyalty Casing (product)

- New york
- Stagnant sales
- Implement a loyalty program in order to grow sales
- Clarifying questions
- Two locations in brooklyn
- Two locations in manhattan
- Coffee sells for around \$5
- Coffee margin around \$3
- Baked goods sell for around \$3
- Baked goods margin around \$2

To prep:

- Clarifying questions
- Current-state sales / margin calculation
- Diagnose stagnant sales
- Segment customers: regulars, occasional, tourists, commuters
- Loyalty program options:
 - stamp card
 - points
 - app
 - phone-number system
 - subscription
 - tiers

For a coffee shop, the loyalty program should primarily move casual customers into regular customers, not heavily reward people who already come every day or random one-time passersby.

Clarifying Question:

- Is this a single coffee shop or a chain? What is the positioning of this store?
- Whats the product mix
- Is maximizing sales our goal
- Is there any constraints to what we can do with the loyalty program
- Is there an existing sale system? Any tracking customers currently?
- Is there a budget for marketing?
- Do we have any data around the drop in growth? Timeframe? specific store?
- Who are our main customers? Or is it a mix?
- Is the goal customer retention, or like increase their visit frequency, or are we trying to push up average order value, or are we trying to perhaps acquire new customers?
- Do we already have customer tracking or membership system like an app?

We have four locations: two in Manhattan and two in Brooklyn. Sales are flat overall. Coffee is \$5 with \$3 margin, baked goods are \$3 with \$2 margin. The goal is to grow sales

So I will first try to define the key metric. Second, segment customers by behavior. Third, identify which segment has the most incremental upside. Fourth, design a practical loyalty mechanism for that segment and measure whether it actually changes behavior.

Sales = number of transactions × average order value

- Define the goal
 - Goal: Revenue = # of customers x frequency of visit x average order/customer
 - Where is the cause of the stagnant sales?
 - If the problem is **traffic (not enough new customers)**: competition, location/not enough foot traffic, seasonality, not marketed enough
 - If the problem is **frequency (not enough repeat customers)**: are regulars coming less often? Retention issues?
 - If the problem is **average order size**: what do customers buy, what are our pricing/bundling
 - Do a margin calculation from the data given
 - Coffee sells for around \$5
 - Coffee margin around \$3
 - Baked goods sell for around \$3
 - Baked goods margin around \$2
 - Gross Margin = coffee units × \$3
 - baked goods units × \$2
 - Margin for coffee = $\frac{3}{5} = 60\%$
 - Margin for Bagels = $\frac{2}{3} = 67\%$

Goal: Increase retention and lifetime value.

For a coffee shop, lifetime value mainly comes from:

- more repeat visits
- higher spend per visit
- longer customer lifespan

6. Customer Segments

I would segment customers by visit frequency and return potential.

For each segment, cover who they are, behavior, need,

Segment 1: Daily regulars

These customers already come almost every day.

- nearby office workers
- students

- commuters
- staff from nearby stores

What they care about

- speed
- consistency
- convenience
- High quality coffee
- maybe mobile preorder

They are valuable customers, but If they already buy coffee every morning, a monetary incentive would mostly reduces margin. And I would prioritize loyalty program that focus on

Things like

- faster pickup
- free customization
- occasional surprise reward
- status/perk
- preorder if possible

Segment 2: Casual / occasional customers

These customers come sometimes, but not habitually.

- People who are just grabbing a coffee in the weekends for afternoon tea
- Or perhaps people who are only grabbing coffee when they need it, like when they need to wake up early some day
- people who rotate between different cafés

They have the intent but not the habit

What they care about

- good reason to come back
- convenient reminder
- Some level of financial incentive
- Business issue

Best incentive

- Coupon for returning
- simple point/stamp system
- Basically systems that reward them for more frequent visits

This is the primary target.

They have enough intent to return, but their behavior is still changeable.

move casual customers into regulars, rather than rewarding people who already come or people who are just passing through once.

So Inactive users would be customers who came once and never returned. So probably mostly passerbys, or tourists that our loyalty program would not be useful for.

So I believe our strategic target should be for casual/occasional customers.

Mechanism:

Option A: Paper punch card. Example: Buy 9 drinks, get the 10th free.

Pros

- Cheapest to mass produce
- It is also very easy to understand the system and its great for a single shop
- And lg customers do lose it very often, so we dont have to actually pay for the cost some times

Cons

- We don't get to track customer data very well
- And customers might not buy into it because keeping these paper cards is a pain

Option B: a mobile app: Point system

- strong tracking
- mobile order
- personalized offers
- push notifications

Cons:

- Hihg customer signup friction

Option 3: Other mechanics

- Subscription models
- Streaks
- Point System
- Tier systems

Revenue calculation for each types of mechanics, what type to behavior can we influence (frequency, order size, etc).

Business Metric - sales

- # of new customers
- Frequency
- Average order size

Northstar metric for the loyalty program

- Average number of orders a month per customer

Two secondary metrics

-

One to two guardrail metrics

- Margin
-

I would pilot this in one Manhattan and one Brooklyn location, compare against the other two locations as control, and measure incremental gross margin, repeat visits, average order value, baked-good attach rate, redemption cost, and checkout friction. If the pilot increases margin after rewards, I would roll it out across all four locations and later consider more advanced personalization or subscription only if the data supports it.

Given this is a 4-location New York coffee shop chain with stagnant sales, I would first diagnose whether sales are stagnant because of flat traffic, weak repeat behavior, low basket size, or location-specific issues.

Assuming the main opportunity is to increase retention and customer lifetime value, I would target **occasional local customers**. Regulars already come often, so heavy discounts risk cannibalization. Tourists and one-time passersby have low repeat potential. Occasional locals are the best segment because they already know the brand but have not built a habit.

My recommendation is to launch a phone-number-based loyalty system across all four locations. Customers earn points or stamps per visit without downloading an app. The core mechanism should be a simple repeat-visit reward, supported by short-window return offers for occasional customers and baked-good attach incentives. For example, after a coffee purchase, an occasional customer might receive “come back within 7 days for \$1 off,” and customers could earn bonus points for adding a baked good.

After customers begin returning more often, the system can add simple progression rewards, such as a free drink after a certain number of visits, or small perks for regular customers like free customization or faster pickup. Over time, if the shop gathers enough customer data, it can add targeted comeback offers for lapsed customers and off-peak incentives for slower periods.

I would measure success through repeat visit rate, second-visit rate, visits per customer per month, casual-to-regular conversion, average order value, reward redemption, and incremental lifetime value net of reward cost. I would also monitor whether the program is just discounting customers who would have come anyway.

So my final recommendation is: start with a lightweight loyalty system focused on getting casual customers to return within a short window, then gradually use the data to retain them as regulars without over-discounting existing daily customers.

Lagging vs leading indicators

- Leading: reward clicks (likely to use it in the future)
- Lagging: retention or lifetime value

Talk about metrics to measure the specific features

They always ask the question open-ended

Practice:

- Answer the question on a high-level while giving some important specific information

ETF Finance Knowledge

What happened on Aug 24th 2015:

Refer to pdf

LULD's role: LULD both helped and complicated the event. It helped by limiting trades outside lower bands and forcing the market to pause when securities hit extreme conditions. But it also operates security by security. An ETF depends on hundreds of underlying stocks plus hedges like futures and related ETFs. If those instruments halt and reopen at different times, the ETF pricing ecosystem becomes asynchronous. So LULD can reduce some bad prints in individual names while still leaving ETF fair-value arbitrage impaired.

Morning: Highest volatility, percentage you can trade is doubled

- >5 → +- 5%
- ETP → 10%
- Leveraged → multiply by leveraged
- Opening and closing → multiply by 2

note*, no early morning double band post Amendment 12

Security	Price bucket	Normal period	9:30–9:45 / 3:35–4:00
Tier 1	> \$3	±5%	±10%
Tier 2	> \$3	±10%	±20%
Tier 1 or Tier 2	\$0.75 to \$3	±20%	±40%
Tier 1 or Tier 2	< \$0.75	± lesser of \$0.15 or 75%	doubled distance
Leveraged ETP	any bucket	base band × leverage	doubled base band × leverage

note*, the 3 dollar is based on previous closing price

Tier	What it includes	Why it matters
Tier 1	S&P 500 stocks, Russell 1000 stocks, selected ETPs	Tighter bands
Tier 2	Other NMS stocks/ETPs, excluding rights/warrants	Wider bands

Examples of tier 1 ETP

- SPY: State street SnP 500 ETF
- QQQ: Invesco QQQ trust (nasdaq 100)
- IWM (iShares Russell 2000 ETF)
- GLD (SPDR Gold Shares)
- TLT (iShares 20+ Year Treasury Bond ETF)

Note* LULD does not cover, rights/warrants, options, futures

Upper Band = Reference Price + Reference Price × percentage

Lower Band = Reference Price – Reference Price × percentage

Reference Price = Arithmetic mean of eligible trades over last 5 min

- SIP continuously calculate a pro-forma reference price, but it is only updated if
(Pro-forma Ref – Current Ref) / Current Ref ≥ 1%
- Once new reference price is in place, it lasts for at least 30 seconds

Opening Reference Price:

- If primary listing exchange opens within 5 minutes, then reference price = opening price
- If it doesn't open, then reference price will be the arithmetic mean of eligible transactions in other exchanges

Limit State:

Entered when the market is at the band

- National Best Offer = Lower Band
- National Best Bid = Upper Band

After entering

- Trading can only happen inside the band or at the band
- If limit state revolves within 15 seconds → continues trading continues
 - For lower Band = 95
 - It is resolved if all ask at 95 are executed or cancelled
 - For upper band = 105
 - It is resolved if all bid at 105 is executed or cancelled

If its not resolved within 15 seconds → Exchange declares 5 minutes trading pause

Straddle State

Trading Pause: No trades occur in that security

- Orders may be entered, cancelled, modified
- Bids/offers may be displayed

Reopening Auction (after 5min trading pause)

- Collects orders
- Publish indicative price
- Try to cross at one price
- Resume continuous trading unless it fails

Both opening and reopening Auction transaction (and single price closing transaction) are not subjected to price collar of LULD

- reopening auction has its own auction Collars
- Matching happens instantaneously for reopening Auction

Auction Failure

- Reasons
 - market order imbalance
 - too much sell pressure
 - too much buy pressure
 - indicative price outside exchange auction collar (LULD bands that triggered the pause) +5% price collar
 - insufficient contra-side liquidity
- If failed: extends for another 5 min
- If failed again: continues process until reopens

Indicative price:

- During Auction, Price at which there is the highest matched volume
- Constantly updated during Auction

If trading pause continue for 10min:

Pre 2019: Other exchanges could resume trading the stock even if the primary exchange had not successfully reopened it

- first Reference Price after pause = last effective Reference Price before pause
- Price band multiply by 3 for 30 seconds after reopening auction finishes

Post Amendment 12: Other exchanges may not trade until primary exchange clears

Last 10 min:

- LULD trading pause is declared at the last 10min, primary exchange does not reopen continuous trading
- It execute the closing auction directly after 5min of trading pause

Trader Script Automation System

- Design an alternative for processing automated scripts from manual traders.
- Manual traders have written macro to automate the manual part of the systems, like order entry or other trade-processing steps. The problem is that these scripts may hit the market directly without proper controls.
- Design something that's safe and governed
- Ask clarifying question
- Define goal: speed vs consistency
- Input outputs, datafields, for scripts, and the market
- Users, guardrails
- Metrics, what determines success
- What are different types of risk, and how to identify and mitigate

Clarifying Questions:

- For equities, ETFs, derivatives?
- What does the script need to do: Order entry only? Cancels/modifies? Hedging?
- Is the system directly sending orders to exchanges, or is it an internal system
- What types of scripts are traders writing? Executing trades? Pulling data?
- Are the scripts run locally or on a cloud?
- Underlying problems? Compliance? Specific risk we are accounting for? Scale issue?
- Who is using it?

Do you mind if i take a min or two to think through this

Walk through your framework

- Definition of goals
- Defining success metrics
- Define different user needs
- Brainstorm different solutions
- Talk about one solution
- And discuss go to market and success metrics

User	What they need
Manual trader	Fast way to automate repetitive workflows
Trader manager	Visibility into what traders are running
Risk team	Ability to approve, monitor, limit, and kill

Compliance	Audit, restricted-list checks, policy enforcement
Exchange/order gateway	Clean, validated, rate-limited orders

Talk about the user in competing interest section

- (1) minimize latency (speed)
- (2) reduce operational risk, error handling (reliability)
- (3) operational accuracy (accuracy)

-

Goals, Metrics, and Tradeoffs (with User Needs)

Discuss the tradeoffs between (1) minimize latency (2) reduce operational risk, error handling and (3) operational accuracy. Address the inherent tension in the system design early on—specifically balancing the trader's need for speed/agility against the firm's absolute need for reliability, compliance, and safety limits.

- Accuracy: Ensure every automated action is precise, properly validated, and strictly risk-checked before entering the market.
- Reliability: Maintain high system stability and error containment.
- Speed: Maintain acceptable execution speed for traders while ensuring thorough logging.
- Consider UDP vs TCP

Clarifying Questions to Ask

- Are the traded instruments equities, ETFs, or derivatives?
- What is the exact scope of the scripts (e.g., order entry only, cancellations, modifications, or active hedging)?
- Are the scripts strictly pulling data, or are they executing trades?
- Is the system sending orders directly to exchanges, or routing through an internal order gateway?
- Are the scripts currently running locally on trader machines or in a cloud environment?
- What specific underlying problem are we solving (e.g., a compliance gap, a specific risk mitigation, or a scalability issue)?

Metrics for Success

- Bad error rejection rate (the volume of problematic orders caught by the system before being sent to the exchange).
- Time to detection (how fast an anomaly is flagged) and time to kill (how fast a rogue script is shut down).
- Adoption rate (the percentage of traders who successfully migrate to the new platform).

Level of automation vs guardrail

- platform is automated and personalized

- UI is e

Also talk about

- Logs

Risk control

- Throttling
- Exceed Risk limits
-

How to get traders to use the platform

- Accessing local APIs
- Build logging to determine what traders are running what scripts
- Then build the centralized platform
- Showcase the trader's scripts is able to be ran in the centralized platform

What do manual traders do:

Observe market

- check current position / risk
- decide whether to trade
- choose instrument, side, size, price, order type
- submit / cancel / modify orders
- monitor fills
- update inventory / hedge / book trade

So the current system would be something like:

Trader macro decides action

- Trader macro sends order directly
- Market

The new system needs to be able to:

- Have the trader define what he wants automated (different parameters)
- Takes the script output and turn it into proposed market actions
- Check the risk before executing the market actions
- Send the order to the market

High-level system design

Scripts being run locally, scripts are different (each trader are building their own script)

- Containers: sandbox into different systems
- Sandboxing: scripts are isolated
- The platform creates a new container, centralize execution of scripts in one place
-
- Compliance team can kill specific scripts

- Pick and choose execution order,

Scripts are similar:

- Deprecate scripts entirely
- Work with the engineer team to build a UI, with features in the UI allowing traders to tweak parameters
- This way there is no risk of unreviewed scripts being ran

Core Architectural Strategy: Containerization & Sandboxing

If traders are running highly customized, individualized scripts, the primary recommendation is to move execution off local machines and into a centralized, containerized environment (e.g., Kubernetes).

Sandboxing for Safety: Each script should live in its own isolated bucket (sandbox). This ensures scripts cannot interact with or corrupt one another.

Error Containment: If a bug is introduced or a script fails, the containerization ensures the error is localized. It will not crash other scripts running in parallel on the orchestrator.

Resource Efficiency: Moving to a centralized kernel/orchestrator allows the system to scale efficiently without wasting idle compute resources.

Things in the Container:

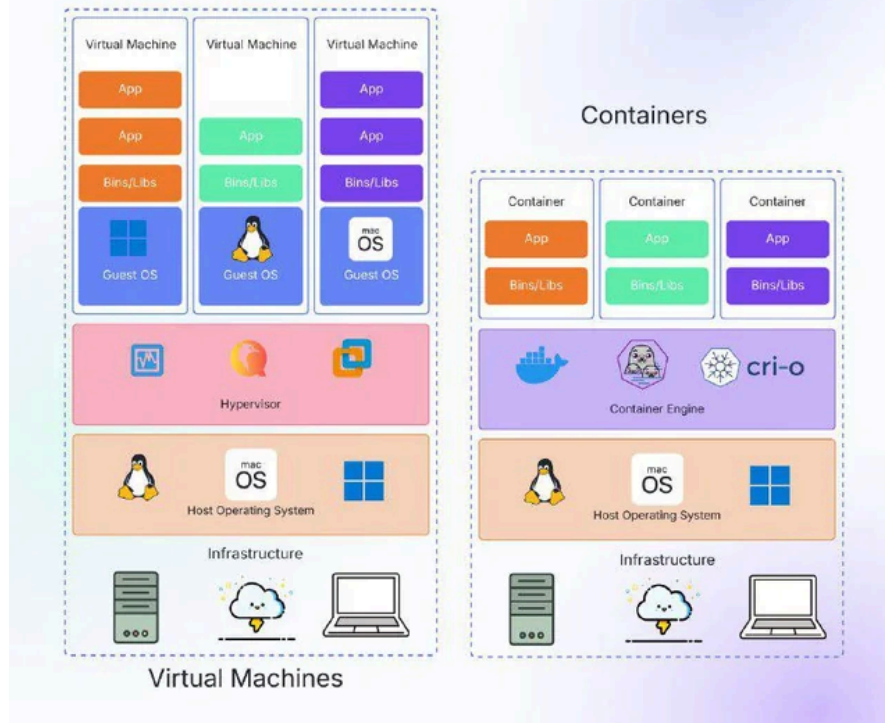
- Trader's Script
- Runtime (eg python 3.14)
- Libraries (numpy, pandas)
- Configuration

Sandboxing creates limits on files, network, CPU/memory resource, permissions

Image: a saved packaged template used to start the container

- One image can start many containers
- Eg image: rsp-strategy-image-v17
- Running container: live instance of rsp-strategy-image-v17

VMs vs CONTAINERS



Resource limit: Max CPU, memory, disk, GPU compute

Permission Limit: block things like

- run as full root admin
- change host network settings
- mount host disks
- access kernel-level features
- escape into the host machine

Kubernetes: manages containers across many servers, it decides

- Where should this container run?
- Is there enough CPU/memory?
- Should this container restart if it crashes?
- How many copies should run?
- Which containers are alive?
- Which machine died?

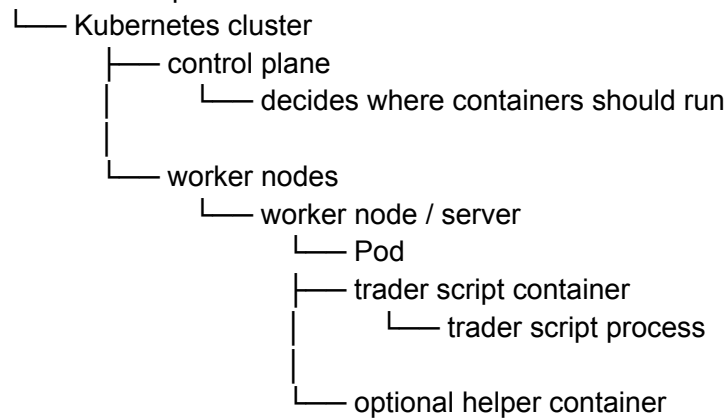
A Kubernetes cluster is a group of machines controlled by kubernetes

Pod: a basic running unit

- One trader script container + a helper container
- Helper container can
 - Record Logs

- Heartbeat
- Credential refresh
- Metrics

Firm internal platform



Namespace: used to label the different function areas of the kubernetes cluster

- Like: Research, ETF trading, equities prod
- A script can be in ETF trading namespace, but it will physically run on server-17
- Each Namespace can have their own policies, such as read live market data, send orders

Firm Paas (platform as a service): trader use the platform to upload their script

- The Pass translate trader's instruction into Kubernetes instructions

Alternative Solution: UI Feature Integration

You should propose a distinct alternative if you determine that the majority of the traders are running very similar scripts.

The 80% Rule: If 75–80% of traders are using scripts to execute the exact same workflows, deprecate the custom scripts entirely.

Engineering Collaboration: Work with the engineering team to build these automated workflows directly into the firm's official trading User Interface (UI).

Benefits: You provide traders with toggles and parameters within a controlled UI. This completely eliminates the risk of unreviewed code running against the market, reduces system redundancy, and frees up traders from having to manually manage their own code.

Risk Controls & Guardrails

A massive part of the system design must focus on what happens when things go wrong. You need to explicitly design for edge cases and system failures:

The "Kill Switch": There must be a mechanism for the Risk/Compliance team (or the trader themselves) to instantly pause or kill all automated scripts tied to a specific trader or desk in the event of a rogue script or "fat finger" error.

Throttling and Rate Limits: You must implement hard limits on message and order volume (e.g., capping a script at a specific number of messages per second). Without rate limits, a localized script loop could flood the exchange gateways or internal endpoints, effectively DDoS-ing your own trading infrastructure.

Pre-Trade Limits: All automated script outputs must still route through standard pre-trade risk checks (maximum order size, position limits, price collars) before hitting the market.

Go-To-Market & Rollout Strategy

You must account for the human element: traders hate friction and are resistant to changing workflows that currently make them money. You cannot simply tell them to use the new system.

Audit Phase: Start by logging local API access to map exactly who is running what scripts. You need visibility into their current behavior before changing it.

Parallel Testing: Prove the new platform works. Run their scripts in the new centralized platform alongside their local setup to demonstrate that execution is reliable, latency is acceptable, and there is no degradation in performance before you officially deprecate local API access.

Data fields:

Order State:

- order_id
- script_id
- symbol
- side
- price
- original_quantity
- filled_quantity
- remaining_quantity
- reserved_quantity
- status
- exchange_sequence_number
- Last_update_time

Order Status (Order lifecycle)

- Pending
- Active
 - Eligible to be matched but filled
- Partially Executed
- Full Executed
- Rejected
- Cancelled (expired)

Inventory Management:

- implied inventory = $\text{OrderFill}(B) - \text{Ordersent}(S) + \text{OrderRejected}(S) + \text{OrderCancelled}(S)$

Inventory Check:

- If quantity < implied Inv, accept, otherwise reject

Used for hedging:

Pre-trade checks (before order enters the book):

- Order size within limits
- Price within acceptable range (price collars)
- Duplicate order detection
- Firm authentication and authorization
- Sufficient capital / exposure limits not exceeded
- Valid ticker symbol and supported order type

Execution checks:

- Price-time priority applied correctly
- Partial fill handling
- Timestamp reconciliation for orders arriving simultaneously (race conditions)

System errors to be aware of:

- Network disconnection — heartbeat checks between exchange and market maker confirm active connection at all times. If heartbeat is lost, orders may be suspended
- Stale market data — if data has not updated within expected window, flag it
- Race condition — cancel request and execution happening at the same nanosecond. Resolved via timestamp priority between market maker and exchange clocks

Human errors to be aware of:

- Fat finger errors (wrong price, wrong quantity)
- Wrong security symbol
- Wrong side (buy instead of sell)
- Duplicate order submission
- Exceeding daily trading limits

Identify abnormal data and prevent risky behavior

Design a system to identify abnormal data and prevent risky behavior in the communication system between market maker and central exchange.

Data Fields we collect

Data type	Examples
Orders	order ID, symbol, side, price, quantity, order type, strategy ID, timestamp
Cancels	order ID, cancel request ID, cancel quantity, timestamp
Exchange responses	accepted, rejected, cancel rejected, cancelled
Fills/trades	execution ID, order ID, price, quantity, remaining quantity, timestamp
Inventory	actual position, reserved quantity, available inventory
Market data	best bid/ask, order book depth, recent trades, volatility
System data	latency, heartbeat status, sequence numbers, dropped messages

Historical data usage

Anomaly Detection System; given months of inventory, orders, and trades data - design a system to find abnormal data

- 1) Define what abnormal data is
 - a) Point anomaly → run mathematical calculation
 - i) Ex fat finger
 - b) Contextual anomaly → compare with other market/system data
 - i) Ex. high trading volume during premarket hours
 - c) Hardware system anomaly → Bug detection, look at internal flows
 - i) Ex. duplicated messages
- 2) Ask if there's a specific bucket to focus on
- 3) Define the goal and who/what you're building for
- 4) Explain solution and implementation
 - a) For each type of anomaly category, explain how you would **detect** that anomaly and then what type of feature/product/system you would build

Ask if i should look for point anomalies or contextual anomalies

How do you design a website to display risk-what would you include?

→ **Build a risk management system from scratch**

(scale up from Identify abnormal data → define abnormal data)

1. Define what risk is.

- a. Financial
- b. Operational
- c. regulatory

- Goal: why do we want to display risk

Who are we building for, why are we building, Goals, importance organizationally

Types of risks → reasons

Dont jump straight into solution

1. Financial Risks

1.1 Fat Finger (stateless)

Risk

- incorrect price
- incorrect quantity
- incorrect order input

Logic checks

- price too far from current market price
- quantity too large
- total order value too large
- invalid symbol
- invalid buy/sell

$\text{Price} \in [\text{market_price} \pm \text{threshold}]$

$\text{Quantity} \leq \text{max_size}$

$\text{Notional} \leq \text{max_notional}$

Data needed

- order: price, quantity, symbol, buy/sell
- market data: best bid / best ask
- internal limits: max order size, max order value

Action

- reject order

1.2 Slippage (stateless & stateful)

Risk

- mismatch in time
- execution price different from expected price

Logic checks

- compare order price vs current order book
- check if order will execute across multiple price levels
- check difference between expected execution price and current price

Slippage: attach timestamp to market data

reject if:

$\text{current_time} - \text{data_timestamp} > \text{threshold}$

Data needed

- order book depth
- best bid / best ask
- recent trades

Action

- reduce order size
- adjust price
- delay order

1.3 Inventory Risk (stateless)

Risk

- too much long position
- too much short position

Logic checks

- current position + order quantity
- check against position limits
- include outstanding orders

Inventory Risk: $|\text{Position} + \text{Pending Orders}| \leq \text{Max Position Limit}$

Sell Quantity $\leq$ Available Inventory

Data needed

- current position
- outstanding orders
- risk limits

Action

- reject order
- reduce quantity

1.4 Overfill (stateless)

Risk

- filling more than intended
- multiple orders filled at same time
- cancel not processed before execution

Logic checks

- include all active orders
- include pending cancel orders
- assume orders can still be executed before cancel

overfill:

track: execution IDs

reject duplicates

Data needed

- outstanding orders
- pending cancels
- exchange feedback (execution, cancel)

Action

- block new orders
- wait for cancel confirmation

2. Operational Risks

2.1 State Inconsistency

Risk

- internal state $\neq$ exchange state

Logic checks

- compare:
 - order state
 - filled quantity
 - remaining quantity

state inconsistency: periodic reconciliation of the positions between exchange and market maker order books, trigger resync if mismatched

consider: frequency of the trading orders, previous history, how the system function, ect.

Data needed

- internal order state
- exchange messages (ACK, execution, cancel)

Action

- pause trading
- resync state

2.2 Race Conditions (only stateful; cannot prevent, resolution & prevention)

Risk

- cancel and execution happen at same time

Logic checks

- cancel request $\neq$ cancelled
- order remains active until confirmed
- execution can still happen

race conditions: all events must be applied in a consistent order

rely on: exchange sequence numbers or authoritative timestamps

Data needed

- cancel request
- execution message
- order state

Action

- wait for confirmation
- update state based on exchange

3. Regulatory Risks

3.1 Shorting Violations

Risk

- selling without inventory

Logic checks

- selling: check inventory
- ensure enough shares to sell

Shorting: $\text{Sell} \leq \text{Position} - \text{Reserved}$ (the ability for MM to borrow)

Data needed

- current inventory
- order quantity

Action

- reject order

3.2 Market Manipulation

Risk

- placing orders not intended to trade

Logic checks

- frequent order placement and cancellation
- large orders cancelled quickly
- low execution vs high order activity

manipulation: detect abnormal pattern:

cancel-to-fill ratio

order lifetime

repeated large fake orders

Data needed

- order frequency
- cancel frequency
- execution history

Action

- block orders
- flag system

4. Core System Checks

4.1 Message Validation

Logic checks

- required fields present
- valid symbol
- valid buy/sell
- quantity > 0

Data needed

- order fields

Action

- reject order

4.2 Duplicate Orders

Risk

- same order sent multiple times

Logic checks

- check order ID uniqueness

Data needed

- order ID
- previous orders

Action

- ignore duplicate

4.3 Kill Switch

Triggers

- position too large
- abnormal behavior
- inconsistent state

Data needed

- position
- system state

Action

- cancel all orders
- stop trading

MACS System